



XSTK Copy - xstk stuff

Xác Xuất Thống Kê (Ho Chi Minh City University of Technology)



Scan to open on Studeersnel

ĐẠI HỌC QUỐC GIA THÀNH PHỐ HỒ CHÍ MINH
TRƯỜNG ĐẠI HỌC BÁCH KHOA
KHOA KHOA HỌC VÀ KỸ THUẬT MÁY TÍNH



BÀI TẬP LỚN XÁC SUẤT THỐNG KÊ
Computer Parts (CPUs and GPUs)

Giảng viên hướng dẫn: Cô Nguyễn Thị Mộng Ngọc
Nhóm thực hiện: Nhóm 10

STT	Họ và tên	MSSV	Tên lớp	Ngành
1	Võ Tiến Nam	2312205	L03	Khoa học Máy tính
2	Nguyễn Hữu Khang	2311449	L03	Khoa học Máy tính
3	Hồ Bắc Nam	2312157	L03	Khoa học Máy tính
4	Nguyễn Tuấn Hiệp	2311011	L03	Khoa học Máy tính
5	Nguyễn Quốc Huy	2113519	L03	Khoa học Máy tính

Thành phố Hồ Chí Minh
Ngày 11 tháng 4 năm 2025





Mục lục

1	Tổng quan dữ liệu	2
1.1	Mô tả chung về bộ dữ liệu	2
1.2	Tập dữ liệu Graphics Processing Units (GPUs)	2
2	Kiến thức nền	3
2.1	Khái niệm về một số đại lượng cơ bản trong thống kê	3
2.2	Ước lượng và kiểm định	4
2.2.1	Ước lượng bằng khoảng tin cậy	4
2.2.2	Kiểm định	5
2.3	Phân tích Phương sai - ANOVA một yếu tố	6
2.4	Hồi quy tuyến tính bội	6
2.4.1	Giới thiệu chung	6
2.4.2	Phương pháp ước lượng hệ số β	7
2.4.3	Hệ số xác định	8
2.4.4	Kiểm định ý nghĩa thống kê của mô hình (kiểm định F)	8
2.4.5	Kiểm định sự phù hợp của các hệ số hồi quy tuyến tính	9
3	Tiền xử lý số liệu	9
3.1	Các thư viện được dùng, các hàm trong ngôn ngữ được dùng để xử lý dữ liệu:	9
3.2	Các thuộc tính của GPUs được chọn để xử lý:	10
3.3	Đọc dữ liệu	11
3.4	Lựa chọn các biến phân tích	11
3.5	Làm sạch dữ liệu không phải số	12
3.6	Xử lý các giá trị N/A	12
3.7	Phương pháp tính giá trị trung bình	14
3.8	Xử lý bước cuối	15
4	Thống kê suy diễn	16
4.1	Kiểm định giả thuyết thống kê trường hợp 2 mẫu	16
4.1.1	Xây dựng giả thuyết	16
4.1.2	Kiểm định phân phối của mẫu	16
4.1.3	Giải quyết bài toán	18
4.1.4	Nhận xét	20
4.1.5	Kết luận	20
4.2	Phân tích Phương sai (ANOVA) 1 Nhân tố	21
4.2.1	Mục tiêu và Yêu cầu Bài toán	21
4.2.2	Giới thiệu về Phân tích Phương sai (ANOVA) 1 Nhân tố	21
4.2.3	Kiểm định và kiểm tra sự đồng nhất phương sai	22
4.2.3.a	Phương pháp Đồ thị Q-Q	22
4.2.3.b	Kiểm định Phương sai Đồng nhất (Levene's Test)	24
4.2.4	Giải theo lý thuyết cho ANOVA 1 nhân tố	25
4.2.5	Giải bằng hàm ANOVA có sẵn của R	27
4.2.6	Kết luận	28



1 Tổng quan dữ liệu

1.1 Mô tả chung về bộ dữ liệu

Bộ dữ liệu này chứa các thông tin về thông tin kỹ thuật chi tiết, khả năng hiển thị, phân loại và tính năng và các thông tin khác của Bộ xử lý trung tâm (Central Processing Unit) và Bộ xử lý đồ họa (Graphics Processing Unit). Đây là các thông tin nhằm mục đích đánh giá hiệu năng, mức tiêu thụ điện năng và khả năng ứng dụng của thiết bị. Dữ liệu trong bộ dữ liệu phần lớn thuộc về các hãng NVIDIA, AMD, Intel; là các công ty tham gia sản xuất các linh kiện này

Bộ dữ liệu chứa hai tệp CSV: `All_GPUs.csv` cho Bộ Xử lý Đồ Họa (GPUs) và `Intel_CPUs.csv` cho Bộ Xử lý Trung Tâm (CPUs). Mỗi bảng có danh sách của các mục riêng, nhưng bộ dữ liệu đều bao gồm những thông tin có thể quy về các nhóm đặc điểm như: tốc độ xử lý, nhiệt độ và công suất tiêu thụ, độ phân giải màn hình và đồ họa tích hợp, công suất tiêu thụ, hỗ trợ ảo hóa và các công nghệ khác, các cổng kết nối và nhiều trường tương tự khác. Tóm lại, hai bộ dữ liệu này cho phép phân tích và so sánh chi tiết giữa GPU và CPU theo nhiều khía cạnh kỹ thuật: hiệu năng, tiêu thụ điện, khả năng kết nối, và thời điểm phát hành. Dù mỗi bảng tập trung vào một loại phần cứng riêng biệt, chúng cùng cung cấp góc nhìn tổng quan cho việc đánh giá, lựa chọn linh kiện phù hợp trong máy tính.

Trong bài tập lớn này, nhóm chúng em đã thống nhất chọn bộ dữ liệu `All_GPUs.csv` cho Bộ xử lý đồ họa (GPUs) làm dữ liệu chính.

1.2 Tệp dữ liệu Graphics Processing Units (GPUs)

Khái niệm: GPU (Graphics Processing Unit) là bộ xử lý chuyên dụng được thiết kế để xử lý các tác vụ liên quan đến đồ họa và hình ảnh, đặc biệt là trong việc hiển thị và dựng hình ảnh 2D, 3D. Ngoài ra, GPU ngày nay còn được sử dụng rộng rãi trong các lĩnh vực như trí tuệ nhân tạo, học máy, tính toán song song và xử lý dữ liệu lớn nhờ khả năng thực hiện nhiều phép tính cùng lúc với hiệu suất cao.

Yêu cầu chung: Sử dụng bộ dữ liệu `All_GPUs.csv`, thực hiện các phương pháp nhằm khắc phục lỗi dữ liệu:

- Khắc phục các giá trị dữ liệu bị khuyết.
- Phát hiện và sửa các giá trị bất thường trong tập dữ liệu.

Tổng quan về bộ dữ liệu: Bộ dữ liệu `All_GPUs.csv` chứa thông tin chi tiết về 3406 mẫu GPU từ nhiều nhà sản xuất khác nhau và 34 biến, trong đó:

- **Biến định lượng:** `Best_Resolution`, `Boost_Clock`, `Core_Speed`, `DVI_Connection`, `Direct_X`, `DisplayPort_Connection`, `HDMI_Connection`, `L2_Cache`, `Max_Power`, `Memory`, `Memory_Bandwidth`, `Memory_Bus`, `Memory_Speed`, `Open_GL`, `PSU`, `Power_Connector`, `Pixel_Rate`, `Process`, `ROPs`, `Release_Price`, `Resolution_WxH`, `Shader`, `TMUs`, `Texture_Rate`, `VGA_Connection`.
- **Biến định tính:** `Architecture`, `Manufacturer`, `Memory_Type`, `Name`, `Dedicated`, `Notebook_GPU`, `Integrated`, `SLI_Crossfire`, `Release_Date`.



2 Kiến thức nền

2.1 Khái niệm về một số đại lượng cơ bản trong thống kê

2.1.0.1 Kỳ vọng (Expectation/Mean) Kỳ vọng của biến ngẫu nhiên X là giá trị trung bình theo xác suất của X . Ký hiệu: $E(X)$.

Công thức tính:

- Đối với biến ngẫu nhiên rời rạc:

$$E(X) = \sum_i X_i p_i$$

- Đối với biến ngẫu nhiên liên tục:

$$E(X) = \int_{-\infty}^{+\infty} x f(x) dx$$

2.1.0.2 Phương sai (Variance) Phương sai của biến ngẫu nhiên X được định nghĩa bằng trung bình bình phương sai lệch giữa biến ngẫu nhiên với kỳ vọng của nó. Ký hiệu: $V(X)$.

Công thức tính:

$$V(X) = E[(X - E(X))^2] \quad \text{hay} \quad V(X) = E(X^2) - [E(X)]^2$$

- Nếu X là biến ngẫu nhiên rời rạc:

$$V(X) = \sum_i [x_i - E(X)]^2 p_i = \sum_i x_i^2 p_i - [E(X)]^2$$

- Nếu X là biến ngẫu nhiên liên tục:

$$V(X) = \int_{-\infty}^{+\infty} [x - E(X)]^2 f(x) dx = \int_{-\infty}^{+\infty} x^2 f(x) dx - [E(X)]^2$$

2.1.0.3 Độ lệch chuẩn (Standard Deviation) Độ lệch chuẩn của biến ngẫu nhiên X là căn bậc hai của phương sai:

$$\sigma_X = \sigma(X) = \sqrt{V(X)}$$

2.1.0.4 Trung vị (Median) Trung vị (median) của biến ngẫu nhiên X , ký hiệu $Md(X)$, là một giá trị sao cho:

$$P(X \leq Md) \geq 0.5 \quad \text{và} \quad P(X \geq Md) \geq 0.5$$

Khi X là biến ngẫu nhiên liên tục thì:

$$P(X \leq Md) = 0.5$$



2.1.0.5 Mốt (Mode) Mốt của biến ngẫu nhiên X (kí hiệu $\text{mod}(X)$) là:

- Đối với biến ngẫu nhiên rời rạc: giá trị của biến ngẫu nhiên X tương ứng với xác suất lớn nhất.
- Đối với biến ngẫu nhiên liên tục: giá trị tương ứng với cực đại của hàm mật độ xác suất.

2.1.0.6 Cực tiểu (Min) Là giá trị nhỏ nhất trong toàn bộ các giá trị của một tập mẫu.

2.1.0.7 Cực đại (Max) Là giá trị lớn nhất trong toàn bộ các giá trị của một tập mẫu.

2.1.0.8 Tứ phân vị (Quartiles) Tứ phân vị là đại lượng mô tả sự phân bố và sự phân tán của tập dữ liệu. Tứ phân vị có 3 giá trị: thứ nhất, thứ hai (trung vị), và thứ ba. Ba giá trị này chia tập dữ liệu (đã sắp xếp từ nhỏ đến lớn) thành 4 phần có số lượng quan sát xấp xỉ bằng nhau:

- Q_1 : tứ phân vị thứ nhất — trung vị của phần dữ liệu nhỏ hơn Q_2 .
- Q_2 : trung vị (median).
- Q_3 : tứ phân vị thứ ba — trung vị của phần dữ liệu lớn hơn Q_2 .

2.1.0.9 Outliers là những giá trị dữ liệu bất thường, nằm ngoài khoảng phân bố chính của tập dữ liệu.

2.2 Ước lượng và kiểm định

Trong bài tập lớn này, nhóm sẽ chọn các phương pháp giải bài toán mẫu như sau:

- Bài toán 1 mẫu: tìm khoảng ước lượng cho trung bình tổng thể.
- Bài toán 2 mẫu: kiểm định 2 mẫu cho 2 trung bình.

2.2.1 Ước lượng bằng khoảng tin cậy

2.2.1.1 Khái niệm:

- Khoảng (c, d) gọi là khoảng ước lượng của θ nếu ta coi $\theta \in (c, d)$.
- Xác suất $P(\theta \in (c, d)) = 1 - \alpha$ gọi là độ tin cậy của khoảng ước lượng.
- Nếu $\hat{\theta}$ là một ước lượng không chệch của θ thì khoảng ước lượng của θ có dạng $(\hat{\theta} - \varepsilon, \hat{\theta} + \varepsilon)$ với $\varepsilon > 0$ gọi là độ chính xác của ước lượng.
- Khi đó:

$$P[\theta \in (\hat{\theta} - \varepsilon, \hat{\theta} + \varepsilon)] = P(|\hat{\theta} - \theta| < \varepsilon) = 1 - \alpha$$



Trung bình tổng thể μ	$(\bar{X} - \varepsilon; \bar{X} + \varepsilon)$	$(\bar{X} - \varepsilon_1; +\infty)$	$(-\infty; \bar{X} + \varepsilon_1)$
Phân phối chuẩn Đã biết σ^2 (2a)	$\varepsilon = Z_{\alpha/2} \cdot \frac{\sigma}{\sqrt{n}}$	$\varepsilon_1 = Z_{\alpha} \cdot \frac{\sigma}{\sqrt{n}}$	$\varepsilon_1 = Z_{\alpha} \cdot \frac{\sigma}{\sqrt{n}}$
Phân phối chuẩn Chưa biết σ^2 (2b)	$\varepsilon = t_{(n-1, \alpha/2)} \cdot \frac{s}{\sqrt{n}}$	$\varepsilon_1 = t_{(n-1, \alpha)} \cdot \frac{s}{\sqrt{n}}$	$\varepsilon_1 = t_{(n-1, \alpha)} \cdot \frac{s}{\sqrt{n}}$
Phân phối tùy ý Mẫu lớn ($n \geq 30$) (2c)	$\varepsilon = Z_{\alpha/2} \cdot \frac{s}{\sqrt{n}}$	$\varepsilon_1 = Z_{\alpha} \cdot \frac{s}{\sqrt{n}}$	$\varepsilon_1 = Z_{\alpha} \cdot \frac{s}{\sqrt{n}}$

2.2.2 Kiểm định

2.2.2.1 Khái niệm: Giả thuyết không H_0 (null hypothesis): là giả thuyết về yếu tố cần kiểm định của tổng thể trong trạng thái bình thường, không chịu tác động của các hiện tượng liên quan. Yếu tố trong H_0 phải được xác định cụ thể, ví dụ:

H_0 : Tỷ lệ nảy mầm của một loại hạt giống là 80%.

2.2.2.2 Giả thuyết đối H_1 (alternative hypothesis): Là một mệnh đề mâu thuẫn với H_0 , thể hiện xu hướng cần kiểm định.

2.2.2.3 Tiêu chuẩn kiểm định: Là hàm thống kê $G = G(X_1, X_2, \dots, X_n; \theta_0)$, xây dựng trên mẫu ngẫu nhiên $V = (X_1, X_2, \dots, X_n)$ và tham số để liên quan đến H_0 , điều kiện giả tạo với thống kê G là nếu H_0 đúng thì quy luật phân phối xác suất của G phải hoàn toàn xác định.

2.2.2.4 Miền bác bỏ (Critical Region, RR): Là miền số thực chứa $P(G \in RR | H_0 \text{ đúng}) = \alpha$, α rất nhỏ (thường 0.05 hoặc 0.01) được gọi là mức ý nghĩa của kiểm định.

2.2.2.5 Miền chấp nhận: Phần bù của miền bác bỏ trong $\mathbb{R}$.

2.2.2.6 Phương pháp:

- Quy tắc kiểm định: Từ mẫu thực nghiệm ta tính được một giá trị cụ thể của tiêu chuẩn kiểm định gọi là giá trị kiểm định thống kê

$$g_T = G(x_1, x_2, \dots, x_n; \theta_0)$$

- Theo nguyên lý xác suất bé, biến cố $G \in RR$ có xác suất nhỏ nếu H_0 đúng thì gần như không thể xảy ra. Do đó:

– Nếu $g_T \in RR$ thì bác bỏ H_0 , tức chấp nhận giả thiết H_1 .



- Nếu $g_T \notin RR$ thì ta vẫn chưa đủ lý lẽ để bác bỏ H_0 , tức chưa chứng minh được H_1 .

2.2.2.7 Kiểm định trung bình bài toán hai mẫu:

TT	Phân bố của tổng thể	Gt H_0	Gt H_1	Miền bác bỏ RR	Tiêu chuẩn kiểm định
(4a)	* 2 mẫu độc lập * $X_1; X_2$ có pp chuẩn. * Đã biết σ_1^2 và σ_2^2	$\mu_1 = \mu_2$	$\mu_1 \neq \mu_2$ $\mu_1 < \mu_2$ $\mu_1 > \mu_2$	$(-\infty; -z_{\alpha/2}) \cup (z_{\alpha/2}; +\infty)$ $(-\infty; -z_{\alpha})$ $(z_{\alpha}; +\infty)$	$Z_{qs} = \frac{\bar{X}_1 - \bar{X}_2}{\sqrt{\frac{\sigma_1^2}{n_1} + \frac{\sigma_2^2}{n_2}}}$ z-test
(4b)	* 2 mẫu độc lập * $X_1; X_2$ có pp chuẩn * Chưa biết $\sigma_1^2; \sigma_2^2$; gt $\sigma_1^2 = \sigma_2^2$ <i>Dấu hiệu quy ước để nhận biết trường hợp (4b) từ mẫu: $\frac{s_1}{s_2} \in \left[\frac{1}{2}; 2\right]$</i>	$\mu_1 = \mu_2$	$\mu_1 \neq \mu_2$ $\mu_1 < \mu_2$ $\mu_1 > \mu_2$	$(-\infty; -t_{\alpha/2; (n_1+n_2-2)}) \cup (t_{\alpha/2; (n_1+n_2-2)}; +\infty)$ $(-\infty; -t_{\alpha; (n_1+n_2-2)})$ $(t_{\alpha; (n_1+n_2-2)}; +\infty)$	$s_p^2 = \frac{(n_1-1)s_1^2 + (n_2-1)s_2^2}{n_1+n_2-2}$ $T_{qs} = \frac{\bar{X}_1 - \bar{X}_2}{\sqrt{\frac{s_p^2}{n_1} + \frac{s_p^2}{n_2}}}$ t-test
(4c)	* 2 mẫu độc lập * $X_1; X_2$ có pp chuẩn * Chưa biết $\sigma_1^2; \sigma_2^2$; có thể giả thiết $\sigma_1^2 \neq \sigma_2^2$ <i>Dấu hiệu quy ước để nhận biết trường hợp (4c) từ mẫu: $\frac{s_1}{s_2} \in \left[\frac{1}{2}; 2\right]$</i>	$\mu_1 = \mu_2$	$\mu_1 \neq \mu_2$ $\mu_1 < \mu_2$ $\mu_1 > \mu_2$	$(-\infty; -t_{\alpha/2; (v)}) \cup (t_{\alpha/2; (v)}; +\infty)$ $(-\infty; -t_{\alpha; (v)})$ $(t_{\alpha; (v)}; +\infty)$ <i>bậc v làm tròn số nguyên</i>	$v = \frac{\left(\frac{s_1^2}{n_1} + \frac{s_2^2}{n_2}\right)^2}{\left(\frac{s_1^2}{n_1}\right)^2 + \left(\frac{s_2^2}{n_2}\right)^2}$ $T_{qs} = \frac{\bar{X}_1 - \bar{X}_2}{\sqrt{\frac{s_1^2}{n_1} + \frac{s_2^2}{n_2}}}$ t-test

2.3 Phân tích Phương sai - ANOVA một yếu tố

2.4 Hồi quy tuyến tính bội

2.4.1 Giới thiệu chung

Hồi quy tuyến tính là một kỹ thuật thống kê dùng để mô hình hóa mối quan hệ giữa một biến phụ thuộc Y (biến mục tiêu) với một hay nhiều biến độc lập X (biến giải thích). Mô hình này giả định rằng mối quan hệ giữa các biến là tuyến tính, tức là có thể được biểu diễn dưới dạng một phương trình đường thẳng trong không gian đa chiều.

Trong hồi quy tuyến tính đơn, chỉ có một biến độc lập được sử dụng để dự đoán biến phụ thuộc. Tuy nhiên, trong hồi quy tuyến tính bội, có nhiều hơn một biến độc lập được sử dụng, đây là phương pháp sẽ được sử dụng trong bài tập lớn này.

Mô hình hồi quy tuyến tính bội liên quan đến biến Y_i và $p-1$ biến dự báo X_i có dạng:

$$Y_i = \beta_0 + \beta_1 X_{i1} + \beta_2 X_{i2} + \dots + \beta_{p-1} X_{(p-1)i} + \epsilon_i$$

Trong đó:

- Y_i : là biến phụ thuộc hay biến mục tiêu tương ứng với quan sát thứ i.
- X_{ik} : là biến giải thích (hay biến giải thích) thứ k cho biến phụ thuộc thứ i, với $k = 1, 2, \dots, p-1$.



- β_k : hệ số hồi quy cho biến độc lập thứ k , trong đó β_0 là hệ số hệ số chặn (intercept).
- $e_i \sim N(0, \sigma^2)$: sai số ngẫu nhiên của mô hình hồi quy hay là các phần dư của dự báo.

Giả định:

- ϵ_i : phân phối chuẩn với trung bình 0 và phương sai không đổi σ^2
- Các biến X_{ik} độc lập với nhau. Kiểm định từng tham số hồi quy tổng thể.
 - **Trường hợp $\beta_k = 0$:** Biến X_{ik} không có ảnh hưởng đáng kể đến biến phụ thuộc Y_i . Khi các biến độc lập khác giữ nguyên, việc thay đổi X_{ik} không làm thay đổi giá trị dự đoán của Y_i .
 - **Trường hợp $\beta_k > 0$:** X_{ik} và Y_i có mối quan hệ thuận chiều. Khi X_{ik} tăng (hoặc giảm), Y_i cũng tăng (hoặc giảm), trong điều kiện các biến độc lập khác không đổi.
 - **Trường hợp $\beta_k < 0$:** X_{ik} và Y_i có mối quan hệ nghịch chiều. Khi X_{ik} tăng (hoặc giảm), Y_i sẽ giảm (hoặc tăng), trong điều kiện các biến độc lập khác không đổi.

Ma trận hồi quy:

$$Y = \begin{bmatrix} Y_1 \\ Y_2 \\ \vdots \\ Y_n \end{bmatrix}, \quad X = \begin{bmatrix} 1 & x_{11} & x_{12} & \cdots & x_{1(p-1)} \\ 1 & x_{21} & x_{22} & \cdots & x_{2(p-1)} \\ \vdots & \vdots & \vdots & \ddots & \vdots \\ 1 & x_{n1} & x_{n2} & \cdots & x_{n(p-1)} \end{bmatrix}, \quad \beta = \begin{bmatrix} \beta_1 \\ \beta_2 \\ \vdots \\ \beta_n \end{bmatrix}, \quad \epsilon = \begin{bmatrix} \epsilon_1 \\ \epsilon_2 \\ \vdots \\ \epsilon_n \end{bmatrix}$$

Với dữ liệu:

$$y = \begin{bmatrix} y_1 \\ y_2 \\ \vdots \\ y_n \end{bmatrix}$$

2.4.2 Phương pháp ước lượng hệ số β

Sử dụng phương pháp bình phương tối thiểu (OLS - Ordinary Least Squares), tức là ước lượng cực tiểu của hàm mất mát:

$$L = \sum_{i=1}^n (y - \hat{y})^2 = \sum_{i=1}^n (y_i - \beta_0 - \sum_{j=1}^{p-1} \beta_j X_{ij})^2$$



Bằng cách tìm giá trị nhỏ nhất của L , ta sẽ tìm được các hệ số hồi quy mẫu phù hợp cho phương trình hồi quy.

Trong khi xây dựng mô hình hồi quy tuyến tính bội, ta cần kiểm tra các giả thuyết như: hàm hồi quy là hàm tuyến tính theo các tham số, sai số ngẫu nhiên độc lập với nhau tuân theo phân phối chuẩn với kỳ vọng bằng 0 và phương sai là $s = \sigma^2$.

2.4.3 Hệ số xác định

Hệ số xác định bội R^2 ($0 \leq R^2 \leq 1$) cho biết mức độ mà mô hình hồi quy giải thích được sự biến thiên của biến phụ thuộc Y_i thông qua các biến độc lập $X_1, X_2, \dots, X_{p-1}$, được tính theo công thức sau:

$$R^2 = \frac{SSR}{SST} = 1 - \frac{SSE}{SST}$$

Trong đó:

- SST (Sum of Squares Total): là tổng bình phương tất cả các sai lệch giữa các giá trị của biến phụ thuộc và giá trị trung bình, được tính theo công thức:

$$SST = \sum_{i=1}^n (y_i - \bar{y})^2$$

- SSR (Sum of Squares in Regression): là tổng bình phương của tất cả các sai lệch giữa các giá trị của biến phụ thuộc Y nhận được từ một hàm hồi quy mẫu và giá trị trung bình của chúng, được tính theo công thức:

$$SSR = \sum_{i=1}^n (\hat{y}_i - \bar{y})^2$$

- SSE (Sum of Squares for Error): dùng để đo sự chênh lệch giữa từng giá trị quan sát với giá trị dự đoán. SSE được xem như sai số do yếu tố ngoài X hoặc do lấy mẫu ngẫu nhiên, được tính theo công thức:

$$SSE = \sum_{i=1}^n (y_i - \hat{y}_i)^2$$

- R^2 càng cao nghĩa là tỷ lệ phần trăm biến thiên của biến phụ thuộc Y so với trung bình của nó do biến X gây ra càng cao.

2.4.4 Kiểm định ý nghĩa thống kê của mô hình (kiểm định F)

Việc kiểm định ý nghĩa thống kê của mô hình hồi quy tuyến tính bội là để xác định xem mô hình có ý nghĩa thống kê hay không, tức là có tồn tại mối quan hệ tuyến tính giữa biến phụ thuộc Y và các biến độc lập $X_1, X_2, \dots, X_{p-1}$ không. Giả thiết biến kiểm định:



- $H_1: \exists \beta_k \neq 0$ (mô hình có ý nghĩa)

Tiêu chuẩn kiểm định:

$$F = \frac{MSR}{MSE} = \frac{R^2/k}{(1 - R^2)/(n - k - 1)}$$

Với mức ý nghĩa α , bác bỏ H_0 nếu:

$$F > F_{\alpha; p-1, n-p}$$

Ý nghĩa: Nếu bác bỏ H_0 , ta kết luận rằng mô hình có ý nghĩa thống kê, tức là có ít nhất một biến độc lập có ảnh hưởng đến biến phụ thuộc.

2.4.5 Kiểm định sự phù hợp của các hệ số hồi quy tuyến tính

Dựa vào kết quả sau khi ước lượng với một mẫu cụ thể, ta có thể đánh giá được mối quan hệ giữa biến phụ thuộc và các biến độc lập trong mô hình một cách tương đối.

Ở mức ý nghĩa α , giả thuyết kiểm định:

- $H_0: \beta_k = 0$
- $H_1: \beta_k \neq 0$

Tiêu chuẩn kiểm định:

$$t = \frac{\hat{\beta}_k}{SE(\hat{\beta}_k)}$$

Trong đó:

- $\hat{\beta}_k$: hệ số hồi quy ước lượng từ mô hình.
- $SE(\hat{\beta}_k)$: sai số chuẩn của hệ số $\hat{\beta}_k$.

Miền bác bỏ:

$$RR = (-\infty; -t_{\alpha/2; n-2}) \cup (t_{\alpha/2; n-2}; +\infty)$$

3 Tiền xử lý số liệu

3.1 Các thư viện được dùng, các hàm trong ngôn ngữ được dùng để xử lý dữ liệu:

Thư viện `<tidyr>` được sử dụng để xử lý và biến đổi dữ liệu. Nó cung cấp các hàm giúp chuyển đổi giữa các cấu trúc dữ liệu, tách hoặc kết hợp các cột, và xử lý giá trị thiếu. Trong bài này ta sử dụng hàm `drop_na()` trong thư viện này để loại bỏ các hàng chứa giá trị thiếu (N/A)

Thư viện `<magrittr>` cung cấp toán tử pipe (thao tác trực tiếp sang hàm tiếp theo theo kiểu trực quan và rõ ràng).

Thư viện `<dplyr>` cung cấp các công cụ thao tác dữ liệu để sử dụng và mạnh mẽ.

Một số hàm của `<dplyr>` được sử dụng trong đoạn code gồm:



- **mutate()**: Trong đoạn code hàm này được sử dụng để làm sạch và chuyển đổi kiểu dữ liệu.
- **across()**: Được sử dụng để áp dụng một hàm nào đó lên tất cả hoặc một nhóm cột phù hợp (trong trường hợp này là cột số).
- **where()**: Kết hợp với **across()** để chỉ định các cột số (**is.numeric**).

3.2 Các thuộc tính của GPUs được chọn để xử lý:

- **Memory_Bus**: chịu trách nhiệm truyền tải dữ liệu giữa bộ nhớ và các bộ phận xử lý của GPU, và độ rộng cũng như tốc độ của nó quyết định trực tiếp băng thông và hiệu suất xử lý của GPU trong các tác vụ đồ họa và tính toán phức tạp.
- **Boost_Clock**: cho biết tốc độ tối đa mà GPU có thể đạt được trong điều kiện cần hiệu suất cao nhất. Thông số này quan trọng vì ảnh hưởng đến hiệu năng tối đa khi chạy các ứng dụng nặng.
- **Core_Speed**: tốc độ lõi cho thấy khả năng xử lý cơ bản của GPU. Tốc độ nhanh hơn thường đồng nghĩa với hiệu năng cao hơn trong các tác vụ đồ họa và xử lý tính toán.
- **Memory**: dung lượng bộ nhớ rất quan trọng vì nó ảnh hưởng đến khả năng lưu trữ dữ liệu tạm thời, tốc độ xử lý dữ liệu và độ phức tạp của đồ họa mà GPU có thể xử lý.
- **Memory_Speed**: tốc độ bộ nhớ cho thấy khả năng truy xuất và xử lý dữ liệu nhanh của bộ nhớ đồ họa, ảnh hưởng trực tiếp đến hiệu năng đồ họa.
- **Memory_Bandwidth**: băng thông bộ nhớ quyết định tốc độ truyền dữ liệu giữa các thành phần bên trong GPU, đóng vai trò quan trọng trong hiệu suất tổng thể của card.
- **Manufacturer**: nhà sản xuất cung cấp thông tin về dòng sản phẩm và mức độ tương thích với phần cứng hoặc phần mềm khác.
- **Shader**: số lượng bộ xử lý shader thể hiện khả năng tính toán và xử lý các hiệu ứng đồ họa. Các ứng dụng nặng về xử lý đồ họa (như chơi game và đồ họa 3D) phụ thuộc rất lớn vào thông số này.
- **Texture_Rate**: tốc độ xử lý texture ảnh hưởng đến khả năng dựng hình ảnh, đặc biệt là trong game và các ứng dụng đồ họa chi tiết.
- **Max_Power**: công suất tối đa giúp xác định mức tiêu thụ năng lượng, ảnh hưởng đến tính hiệu quả và yêu cầu về hệ thống tản nhiệt và nguồn điện.
- **Process**: công nghệ sản xuất ảnh hưởng đến kích thước chip và mức tiêu thụ năng lượng, từ đó tác động đến hiệu năng và khả năng tản nhiệt của GPU.



3.3 Đọc dữ liệu

```
# Đọc dữ liệu  
all_gpus <- read.csv("All_GPUs.csv")
```

- Hàm `read.csv()` dùng để đọc dữ liệu từ một tệp CSV vào R và lưu dưới dạng một `data.frame`.
- Trong R, `data.frame` là một cấu trúc dữ liệu phổ biến dùng để lưu trữ và xử lý dữ liệu dạng bảng (table). Nó giống như một bảng tính trong Excel hoặc một bảng trong cơ sở dữ liệu.
- Mỗi `data.frame` bao gồm các cột, trong đó mỗi cột có thể chứa dữ liệu kiểu khác nhau (số, chuỗi, yếu tố, v.v.), nhưng tất cả các cột phải có cùng độ dài.

3.4 Lựa chọn các biến phân tích

```
parameter_gpus <- c(  
  "Boost_Clock", "Core_Speed", "Manufacturer", "Memory", "Shader",  
  ↪ "Memory_Speed", "Memory_Bandwidth", "Texture_Rate", "Process",  
  ↪ "Max_Power", "Memory_Bus"  
)  
# Lựa chọn các biến từ dữ liệu gốc  
all_gpus_news <- all_gpus[, parameter_gpus]  
  
# Xem dữ liệu đã chọn  
view(all_gpus_news)
```

Giải thích các hàm trong đoạn mã:

- `c(...)`: là hàm tạo một vector trong R. Trong trường hợp này, nó được dùng để liệt kê các tên biến (cột) cần chọn từ bộ dữ liệu gốc.
- `parameter_gpus <- c(...)`: gán danh sách các tên biến cần phân tích vào một đối tượng có tên `parameter_gpus`.
- `all_gpus[, parameter_gpus]`: chọn các cột có tên nằm trong vector `parameter_gpus` từ dataframe `all_gpus`. Đây là cú pháp chọn cột trong R bằng tên, thay vì chỉ số.
- `all_gpus_news <- all_gpus[, parameter_gpus]`: gán kết quả lọc cột ở trên vào một dataframe mới có tên `all_gpus_news`.
- `view(all_gpus_news)`: hiển thị dataframe `all_gpus_news` trong một cửa sổ riêng để người dùng có thể xem dữ liệu một cách trực quan (chỉ hoạt động trong môi trường IDE như RStudio).



3.5 Làm sạch dữ liệu không phải số

Việc làm sạch dữ liệu là một bước rất quan trọng trong quá trình phân tích dữ liệu, bởi vì dữ liệu thực tế thường chứa nhiều lỗi, thiếu sót, hoặc định dạng không phù hợp, ví dụ như dữ liệu có ký tự không phải số trong các cột số (có thể là thứ nguyên như trong các cột "Boost_Clock", "Memory", "Max_Power").

```
#Làm sạch và chuyển đổi dữ liệu
all_gpus_news <- all_gpus_news %>%
  mutate(
    Memory_Bus = as.numeric(gsub("[^0-9.]", "", Memory_Bus)),
    Boost_Clock = as.numeric(gsub("[^0-9.]", "", Boost_Clock)),
    Core_Speed = as.numeric(gsub("[^0-9.]", "", Core_Speed)),
    Manufacturer = as.factor(Manufacturer),
    Memory = as.numeric(gsub("[^0-9.]", "", Memory)),
    Shader = as.numeric(gsub("[^0-9.]", "", Shader)),
    Memory_Speed = as.numeric(gsub("[^0-9.]", "", Memory_Speed)),
    Memory_Bandwidth = as.numeric(gsub("[^0-9.]", "",
    ↪ Memory_Bandwidth)),
    Texture_Rate = as.numeric(gsub("[^0-9.]", "", Texture_Rate)),
    Process = as.numeric(gsub("[^0-9.]", "", Process)),
    Max_Power = as.numeric(gsub("[^0-9.]", "", Max_Power))
  )
```

3.5.0.1 Giải thích các hàm trong đoạn mã trên:

- `mutate(...)`: Dùng để tạo hoặc thay đổi các cột trong một dataframe. Trong trường hợp này, các cột đang được chuyển đổi giá trị.
- `gsub("[^0-9.]", "", tên_cột)`: Hàm này loại bỏ tất cả các ký tự không phải là chữ số hoặc dấu chấm thập phân.
 - `[^0-9.]` là biểu thức chính quy (regex), nghĩa là “không phải là các ký tự từ 0 đến 9 hoặc dấu chấm”.
 - Ví dụ: nếu cột chứa chuỗi "1200 MHz" thì kết quả sau `gsub` sẽ là "1200".
- `as.numeric(...)`: Chuyển đổi chuỗi (string) kết quả từ `gsub` sang kiểu dữ liệu số thực (numeric) để có thể phân tích thống kê.
- `as.factor(...)`: Dùng để chuyển cột định danh (ví dụ tên nhà sản xuất) thành kiểu `factor` – một kiểu dữ liệu định danh trong R, dùng cho phân tích thống kê rời rạc hoặc phân loại.

3.6 Xử lý các giá trị N/A

```
# Tính tỷ lệ NA của từng cột
na_ratios <- colSums(is.na(all_gpus_news)) / nrow(all_gpus_news)
```



```
# Loại bỏ các cột có tỷ lệ NA > 50%
all_gpu_news <- all_gpu_news[, na_ratios <= 0.5]

# Loại các dòng có NA ở các cột có tỷ lệ NA < 6%
delete_NA_col <- names(na_ratios[na_ratios < 0.06])
all_gpu_news <- all_gpu_news %>% drop_na(all_of(delete_NA_col))
```

Một số cột có thể có nhiều giá trị bị thiếu (N/A), nếu giữ lại quá nhiều các thông số có giá trị N/A (lớn hơn 50%) có thể gây sai lệch khi tính toán thống kê, vì vậy chúng ta cần phải loại bỏ các thông số này.

Phương án được chọn đó là:

- Loại bỏ cột nếu hơn 50% giá trị bị thiếu.
- Với cột có ít NA (dưới 6%), loại bỏ dòng chứa NA.
- Với các cột còn lại, thay NA bằng giá trị trung bình của cột đó.

Giải thích các hàm R sử dụng:

- `is.na(...)`: Trả về giá trị `TRUE` nếu phần tử là NA (giá trị bị thiếu).
- `colSums(...)`: Tính tổng số lượng NA theo từng cột (do `is.na` trả về ma trận `TRUE/FALSE`).
- `nrow(...)`: Trả về số dòng trong bảng dữ liệu, dùng để tính tỷ lệ phần trăm NA.
- `na_ratios <- colSums(is.na(...)) / nrow(...)`: Tính tỷ lệ giá trị NA của mỗi cột.
- `all_gpu_news[, na_ratios <= 0.5]`: Lọc giữ lại các cột có tỷ lệ NA nhỏ hơn hoặc bằng 50%.
- `names(...)`: Lấy tên các cột có tỷ lệ NA nhỏ hơn 6%.
- `drop_na(all_of(...))`: Loại bỏ toàn bộ dòng chứa NA tại các cột được chỉ định.

Việc chọn ngưỡng 6% là một cách tiếp cận thực tế để xử lý các cột có lượng nhỏ giá trị thiếu, nhằm giảm thiểu việc mất mát dữ liệu không cần thiết. Nếu tỷ lệ NA quá thấp (dưới 6%), việc loại bỏ một số dòng sẽ không ảnh hưởng nhiều đến tính đại diện của toàn bộ tập dữ liệu, nhưng lại giúp đảm bảo tính toàn vẹn khi tính toán thống kê và huấn luyện mô hình sau này.

Khi in ra `na_ratios`, ta được các kết quả sau:

Boost_Clock	Core_Speed	Manufacturer	Memory
0.57545508	0.27480916	0.00000000	0.12331180
Shader	Memory_Speed	Memory_Bandwidth	Texture_Rate
0.03141515	0.03082795	0.03552554	0.15971814
Process	Max_Power	Memory_Bus	
0.13593658	0.18349971	0.01820317	



Vì "Boost_Clock" có tỉ lệ N/A chiếm đến 0,575 nên giá trị trên cần bị loại bỏ.
Các cột có tỉ lệ N/A nhỏ hơn 6% sẽ được giữ lại và các giá trị N/A trong cột này thay vì sử dụng phương pháp giá trị trung bình:

```
[1] "Manufacturer" "Shader" "Memory_Speed" "Memory_Bandwidth"
[5] "Memory_Bus"
>
```

Kết quả cuối cùng được lưu vào file All_GPUs_Cleaned.csv:

	A	B	C	D	E	F	G	H	I	J	K	L	M	N	O
	Core Speed	Manufactu...	Memory	Sha...	Memory Spe...	Memory Bandwi...	Texture Rate	Process	Max Power	Memory ...					
1	738	Nvidia	1024	4	1000	64	47	55	141	256					
2	942.416425918283	AMD	512	4	828	106	12	80	215	512					
3	942.416425918283	AMD	512	4	800	51.2	10	80	200	256					
4	942.416425918283	AMD	256	4	1150	36.8	7	65	127.8515625	128					
5	942.416425918283	AMD	256	4	700	22.4	6	65	45	128					
6	942.416425918283	AMD	256	4	1100	35.2	6	65	50	128					
7	870	AMD	2048	4.1	1050	134.4	35	55	190	256					
8	942.416425918283	AMD	256	4	800	51.2	7	80	150	256					
9	942.416425918283	AMD	2048	5	1250	160	62	28	150	256					
10	942.416425918283	AMD	64	1	366	2.9	90.8544802867384	31.9700034879665	32	64					
11	942.416425918283	Nvidia	128	1.3	325	5.2	2	150	127.8515625	128					
12	650	Nvidia	6144	5	925	177.6	36	40	250	384					
13	705	Nvidia	5120	5	1050	168	90.8544802867384	28	225	320					
14	706	Nvidia	12288	5	1502	288.4	169	28	245	384					
15	1050	AMD	3072	5	900	57.6	62	28	150	128					
16	942.416425918283	Nvidia	8192	5	1250	320	90.8544802867384	28	300	512					
17	732	Nvidia	6144	5	1300	249.6	90.8544802867384	28	235	384					
18	300	Intel	2909.81968911917	5	1067	34.1	8	14	28	128					
19	942.416425918283	AMD	256	2	730	23.4	3	150	60	256					
20	676	Nvidia	6144	5	760	1.6	90.8544802867384	28	225	384					

3.7 Phương pháp tính giá trị trung bình

Với các cột có giá trị bị thiếu (NA) nhưng tỷ lệ NA không quá lớn (không bị loại bỏ hoàn toàn), ta áp dụng phương pháp thay thế các giá trị thiếu bằng **giá trị trung bình của cột tương ứng**. Đây là một kỹ thuật đơn giản nhưng hiệu quả trong việc làm sạch dữ liệu, giúp hoàn thiện tập dữ liệu mà không làm mất đi quá nhiều thông tin.

Phương pháp này không chỉ giúp giữ lại các thông tin quan trọng thay vì loại bỏ toàn bộ dòng hoặc cột, mà còn góp phần **giảm thiểu sai lệch** trong quá trình phân tích thống kê.

Trong R, việc này được thực hiện như sau:

```
# Hàm tính giá trị trung bình cho từng cột nếu là số
meanCol <- function(x) {
  if (is.numeric(x)) {
    mean(x, na.rm = TRUE)
  } else {
    NA
  }
}
```



```
}  
}  
  
# Tính trung bình cho tất cả các cột  
mean_values <- sapply(all_gpus_news, meanCol)  
  
# Thay thế NA bằng giá trị trung bình tương ứng  
all_gpus_news <- all_gpus_news %>%  
  mutate(across(where(is.numeric), ~ ifelse(is.na(.),  
    ↪ mean_values[cur_column()], .)))
```

- `sapply(...)`: Đây là một hàm trong R dùng để áp dụng một hàm cụ thể cho từng phần tử trong một đối tượng (ở đây là các cột của `all_gpus_news`). Trong trường hợp này, nó được sử dụng để áp dụng hàm `meanCol` cho tất cả các cột và trả về kết quả dưới dạng một vector.
- `meanCol`: Đây là một hàm tự định nghĩa có nhiệm vụ tính giá trị trung bình cho các cột có kiểu dữ liệu số (`numeric`). Hàm kiểm tra kiểu dữ liệu và nếu đúng là số thì sẽ sử dụng `mean(x, na.rm = TRUE)` để tính trung bình, trong đó `na.rm = TRUE` có nghĩa là bỏ qua các giá trị bị thiếu (NA).
- `mean(x, na.rm = TRUE)`: Là hàm tính giá trị trung bình trong R, với tùy chọn `na.rm = TRUE` giúp loại bỏ các giá trị NA ra khỏi phép tính để tránh gây lỗi hoặc sai số.
- `mutate(across(...))`: Câu lệnh này đến từ thư viện `dplyr`, dùng để cập nhật giá trị cho các cột. Cụ thể ở đây, nó được dùng để thay thế các giá trị NA trong tất cả các cột số bằng giá trị trung bình tương ứng đã được tính từ trước.

3.8 Xử lý bước cuối

Xóa các giá trị N/A còn lại, đặt lại chỉ số hàng, và hiển thị dữ liệu đã được làm sạch.

```
# Loại bỏ các hàng còn lại chứa N/A  
all_gpus_news_clean <- na.omit(all_gpus_news)  
  
# Đặt lại chỉ số hàng  
rownames(all_gpus_news_clean) <- NULL  
  
# Hiển thị dữ liệu đã được xử lý  
View(all_gpus_news_clean)
```

- `na.omit(...)`: Loại bỏ toàn bộ các dòng còn chứa giá trị bị thiếu (NA) còn sót lại trong bảng dữ liệu.
- `rownames(...)`: Là hàm trong R dùng để truy cập hoặc thay đổi chỉ số (index) của các hàng trong một `data.frame`. Câu lệnh `rownames(all_gpus_news_clean) <- NULL` có tác dụng xóa bỏ các chỉ số hàng hiện tại.



dụng thiết lập lại chỉ số hàng bắt đầu từ 1 và liên tục, đặc biệt cần thiết sau khi dữ liệu bị lọc nhiều dòng.

- **View(...)**: Hiển thị bảng dữ liệu cuối cùng đã được làm sạch trong giao diện trực quan của RStudio để người dùng kiểm tra lại kết quả.

4 Thống kê suy diễn

4.1 Kiểm định giả thuyết thống kê trường hợp 2 mẫu

Yêu cầu bài toán

Dựa vào tập dữ liệu gồm các GPU thuộc nhiều nhà sản xuất khác nhau, hãy lọc ra hai mẫu là hai nhà sản xuất liên tiếp lần lượt là AMD và Nvidia (dựa vào biến `Manufacturer`). Từ đó so sánh giá trị công suất tiêu thụ tối đa (`Max_power`) trung bình của nhà sản xuất Nvidia và nhà sản xuất AMD. Với mức ý nghĩa 5%, có thể coi giá trị `Max_power` trung bình của các GPU được sản xuất bởi Nvidia lớn hơn các GPU được sản xuất bởi AMD không?

4.1.1 Xây dựng giả thuyết

- Đặt μ_1 là trung bình các giá trị `Max_Power` của hãng Nvidia.
- Đặt μ_2 là trung bình các giá trị `Max_Power` của hãng AMD.
- H_0 : Trung bình Max Power của Nvidia không lớn hơn AMD ($\mu_1 \leq \mu_2$)
- H_1 : Trung bình Max Power của Nvidia lớn hơn AMD ($\mu_1 > \mu_2$)

Kiểm định với mức ý nghĩa $\alpha = 5\%$

4.1.2 Kiểm định phân phối của mẫu

- Đọc dữ liệu đã được làm sạch và lọc từ bước Tiền xử lí dữ liệu.
- Chia dữ liệu thành 2 mẫu theo hai nhà sản xuất Nvidia và AMD:

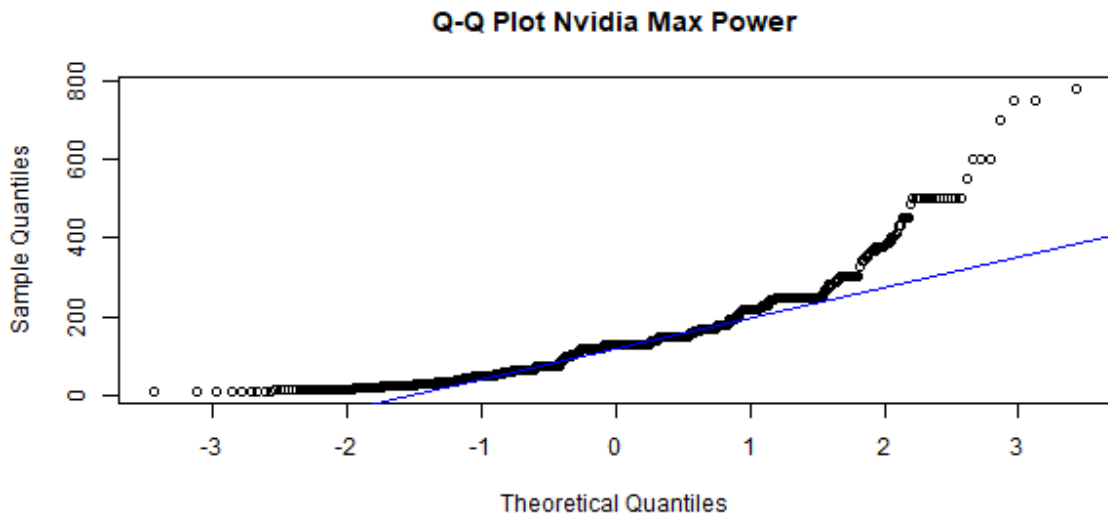
```
1 nvidia <- all_gpus %>% filter((Manufacturer == "Nvidia"))  
2 amd <- all_gpus %>% filter((Manufacturer == "AMD"))
```

- Kiểm tra giá trị `Max_Power` của 2 hãng Nvidia và AMD có tuân theo phân phối chuẩn không bằng 2 cách:
 - Phương pháp đồ thị: Sử dụng đồ thị Q-Q (Quantile - Quantile).
 - Phương pháp thống kê: Sử dụng kiểm định Shapiro-Wilk

Sử dụng đồ thị Q-Q: Vẽ đồ thị cho biến `Max_Power` của 2 nhà sản xuất Nvidia và AMD:

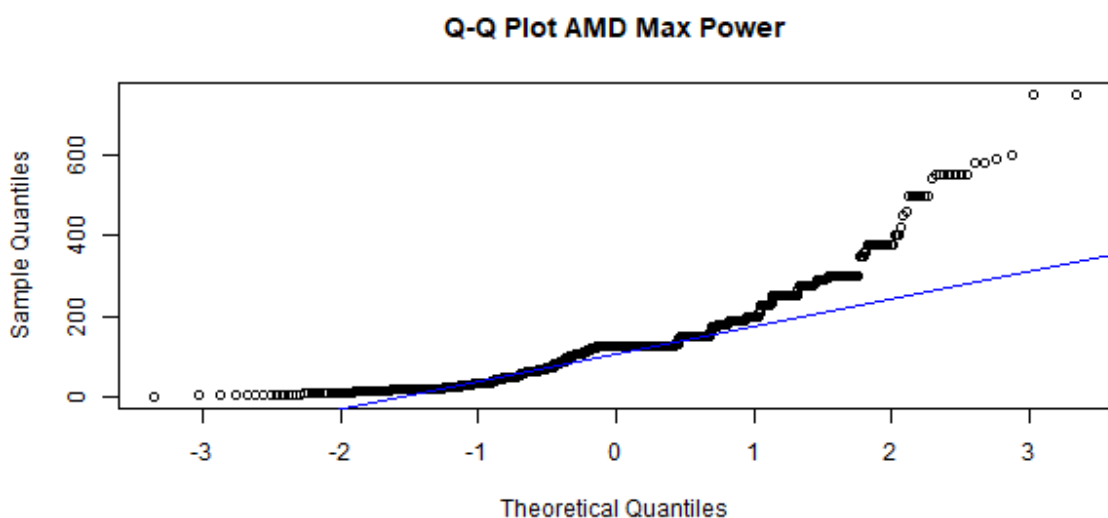


```
1 qqnorm(nvidia$Max_Power, main = "Q-Q Plot Nvidia Max Power")
2 qqline(nvidia$Max_Power, col = "blue")
```



AMD:

```
1 qqnorm(amd$Max_Power, main = "Q-Q Plot AMD Max Power")
2 qqline(amd$Max_Power, col = "blue")
```



Từ đồ thị, có thể thấy cả hai đường phân phối Max_Power của hãng Nvidia và AMD đều khá lệch so với đường kì vọng phân phối chuẩn. Do đó có thể kết luận là các giá trị Max_Power của hai hãng đều không tuân theo phân phối chuẩn.



Sử dụng kiểm định Shapiro-Wilk: Để kiểm định được kết quả đúng, ta sử dụng sử dụng kiểm định Shapiro-Wilk với giả thuyết kiểm định:

- H_0 : Biến kiểm định `Max_Power` có tuân theo phân phối chuẩn
- H_1 : Biến kiểm định `Max_Power` không tuân theo phân phối chuẩn

Code R:

```
1 shapiro.test(nvidia$Max_Power)
2 shapiro.test(amd$Max_Power)
```

Kết quả:

```
Shapiro-wilk normality test

data:  nvidia$Max_Power
W = 0.86425, p-value < 2.2e-16
```

```
Shapiro-wilk normality test

data:  amd$Max_Power
W = 0.85443, p-value < 2.2e-16
```

Sau khi kiểm định biến `Max_Power` của hai nhà sản xuất bằng kiểm định Shapiro-Wilk, ta thu được kết quả là $p\text{-value} \ll \alpha = 0.05$. Do đó, ta bác bỏ giả thuyết H_0 là biến kiểm định `Max_Power` tuân theo phân phối chuẩn, và chấp nhận giả thuyết H_1 là biến này của cả hai nhà sản xuất Nvidia và AMD đều không tuân theo phân phối chuẩn.

4.1.3 Giải quyết bài toán

- Từ dữ liệu ban đầu và các kiểm định đã thực hiện, ta có thể rút ra các nhận xét sau về bài toán kiểm định 2 mẫu:
 - Đây là 2 mẫu độc lập (hai nhà sản xuất khác nhau).
 - Biến `Max_Power` không tuân theo phân phối chuẩn mà có phân phối tùy ý.
 - Kích thước mẫu lớn (cả 2 mẫu đều có số lượng lớn hơn 30).
 - Chưa biết phương sai của cả hai mẫu.
- Theo định lý giới hạn trung tâm, khi kích thước mẫu đủ lớn, phân phối trung bình của các biến ngẫu nhiên sẽ tiệm cận phân phối chuẩn, cụ thể, $Z = \frac{\bar{X} - \mu_0}{S/\sqrt{n}}$ sẽ hội tụ về phân phối chuẩn hóa $Z \sim N(0, 1)$. Do đó, có thể coi trung bình của giá trị `Max_Power` tuân theo phân phối chuẩn và sử dụng kiểm định z (Z-test) để giải quyết bài toán, với giá trị phương sai tổng thể được thay thế bằng phương sai mẫu.



- Để giải quyết bài toán, nhóm sẽ dùng lý thuyết để tự tính và sử dụng z.test của R để kiểm tra kết quả.

Giải quyết theo cơ sở lý thuyết:

Sử dụng hàm `qnorm()` để tính giá trị của `z_score`, từ đó rút ra được miền bác bỏ của giả thuyết:

```
1 alpha <- 0.05
2 z_score <- qnorm(1 - alpha)
3 print(z_score)
```

Kết quả thu được:

```
[1] 1.644854
```

Với giá trị `z_score` là 1.644854, ta rút ra được miền bác bỏ: $RR = (1.644854; +\infty)$

Tính toán giá trị trung bình và phương sai mẫu cho hãng Nvidia:

```
1 n_nvidia <- length(nvidia$Max_Power)
2 mean_nvidia <- mean(nvidia$Max_Power)
3 sd_nvidia <- sd(nvidia$Max_Power)
```

Kết quả:

```
n_nvidia mean_nvidia sd_nvidia
1684      134.7765  93.81523
```

Tính toán giá trị trung bình và phương sai mẫu cho hãng AMD:

```
1 n_amd <- length(amd$Max_Power)
2 mean_amd <- mean(amd$Max_Power)
3 sd_amd <- sd(amd$Max_Power)
```

Kết quả:

```
n_amd mean_amd sd_amd
1225 130.8899 100.4345
```

Từ cơ sở lý thuyết thực hiện tính giá trị thống kê kiểm định theo công thức:

$$z = \frac{\bar{x}_1 - \bar{x}_2}{\sqrt{\frac{s_1^2}{n_1} + \frac{s_2^2}{n_2}}}$$

Code R:

```
1 n_amd <- length(amd$Max_Power)
2 mean_amd <- mean(amd$Max_Power)
3 sd_amd <- sd(amd$Max_Power)
```



Kết quả:

$Z = 1.05934462217387$

Với kết quả thu được, ta có giá trị thống kê kiểm định: $z = 1.0593 \notin RR = (1.644854; +\infty)$ Để kiểm tra kết quả tính toán trước khi kết luận, ta sẽ sử dụng hàm `z.test` được cung cấp trong ngôn ngữ R để thực hiện tính toán kết quả một lần nữa:

```
1  z.test(  
2    x = nvidia$Max_Power,  
3    y = amd$Max_Power,  
4    mu = 0,  
5    sigma.x = sd_nvidia,  
6    sigma.y = sd_amd,  
7    alternative = "greater",  
8    conf.level = 0.95  
9  )
```

Kết quả thu được:

```
Two-sample z-Test  
  
data: nvidia$Max_Power and amd$Max_Power  
z = 1.0593, p-value = 0.1447  
alternative hypothesis: true difference in means is greater than 0  
95 percent confidence interval:  
-2.148171      NA  
sample estimates:  
mean of x mean of y  
134.7765 130.8899
```

4.1.4 Nhận xét

- Với kết quả thu được từ việc tính toán thủ công dựa vào lý thuyết và kiểm tra kết quả tính toán dựa vào hàm `z.test`, ta thu được kết quả giống nhau là $z = 1.0593 \notin RR = (1.644854; +\infty)$
- Do đó, ta không thể bác bỏ giả thuyết H_0 mà phải chấp nhận là theo tổng thể, giá trị `Max_Power` trung bình của các GPU do hãng Nvidia sản xuất không thể lớn hơn các GPU được sản xuất bởi hãng AMD.

4.1.5 Kết luận

Qua bài toán trên, nhóm đã có cái nhìn rõ hơn về một bài toán kiểm định giả thuyết thống kê 2 mẫu, sử dụng được ngôn ngữ R cơ bản để giải quyết bài toán dựa trên các hàm có sẵn và tính toán thủ công, từ đó đưa ra một kết luận cho một bài toán kiểm định.



4.2 Phân tích Phương sai (ANOVA) 1 Nhân tố

4.2.1 Mục tiêu và Yêu cầu Bài toán

Chúng ta sẽ thực hiện phân tích phương sai (ANOVA 1 nhân tố) để kiểm tra sự khác biệt giữa các nhà sản xuất GPU, bao gồm **Nvidia**, **AMD**, **Intel**, và **ATI**, về **tốc độ bộ nhớ (Memory Speed)**. Mục tiêu là xác định xem có sự khác biệt có ý nghĩa thống kê giữa các nhà sản xuất này hay không.

Giả thuyết kiểm định được xây dựng như sau:

- μ_1 : Trung bình tốc độ bộ nhớ của các GPU sản xuất bởi Nvidia.
- μ_2 : Trung bình tốc độ bộ nhớ của các GPU sản xuất bởi AMD.
- μ_3 : Trung bình tốc độ bộ nhớ của các GPU sản xuất bởi Intel.
- μ_4 : Trung bình tốc độ bộ nhớ của các GPU sản xuất bởi ATI.
- Giả thuyết không (H_0): Các trung bình tốc độ bộ nhớ của các nhóm là bằng nhau, tức là $\mu_1 = \mu_2 = \mu_3 = \mu_4$.
- Giả thuyết thay thế (H_1): Có ít nhất một sự khác biệt giữa các trung bình tốc độ bộ nhớ của các nhóm.

Phân tích sẽ được thực hiện với mức ý nghĩa $\alpha = 5\%$.

4.2.2 Giới thiệu về Phân tích Phương sai (ANOVA) 1 Nhân tố

- Phân tích phương sai ANOVA 1 nhân tố được sử dụng để kiểm tra sự khác biệt giữa các nhóm dựa trên một biến định lượng (ở đây là **Memory Speed**). Trong trường hợp này, chúng ta kiểm tra xem có sự khác biệt về tốc độ bộ nhớ giữa bốn nhà sản xuất **Nvidia**, **AMD**, **Intel**, và **ATI**.
- ANOVA 1 nhân tố so sánh sự biến thiên giữa các nhóm (sự khác biệt giữa các trung bình nhóm) với sự biến thiên trong mỗi nhóm (sai số ngẫu nhiên trong mỗi nhóm).
- Các bậc tự do trong phân tích ANOVA được chia thành các thành phần sau:
 - **df_{between} (Bậc tự do giữa nhóm)**: Là số bậc tự do được sử dụng để đo lường sự khác biệt giữa các trung bình nhóm.

$$df_{\text{between}} = k - 1$$

với $k = 4$ (Nvidia, AMD, Intel, ATI).

- **df_{within} (Bậc tự do trong nhóm)**: Là số bậc tự do được sử dụng để đo lường sự biến thiên trong mỗi nhóm.

$$df_{\text{within}} = n - k$$

với n là tổng số quan sát và k là số nhóm.



- **df_{total}** (Bậc tự do tổng thể): Là tổng số bậc tự do của toàn bộ dữ liệu.

$$df_{\text{total}} = n - 1$$

- ANOVA tính toán tỷ lệ giữa **MS_{between}** (Mean Square Between) và **MS_{within}** (Mean Square Within), được gọi là giá trị F. Nếu giá trị F lớn, điều đó cho thấy sự khác biệt giữa các nhóm là có ý nghĩa.

$$F = \frac{MS_{\text{between}}}{MS_{\text{within}}}$$

Trong đó:

- $MS_{\text{between}} = \frac{SS_{\text{between}}}{df_{\text{between}}}$
- $MS_{\text{within}} = \frac{SS_{\text{within}}}{df_{\text{within}}}$

Các giá trị **SS_{between}** và **SS_{within}** là tổng bình phương sai số giữa các nhóm và trong các nhóm.

4.2.3 Kiểm định và kiểm tra sự đồng nhất phương sai

Trước khi thực hiện phân tích ANOVA, chúng ta cần kiểm tra sự đồng nhất phương sai và tính phân phối chuẩn của các nhóm. Để làm điều này, ta sử dụng hai phương pháp kiểm tra: **Phương pháp đồ thị Q-Q** và **Kiểm định Phương sai Đồng nhất (Levene's Test)**.

- **Đọc dữ liệu:** Để chuẩn bị cho phân tích, chúng ta sẽ lọc dữ liệu theo từng nhóm nhà sản xuất:

```
1 # Lọc các nhóm dữ liệu theo từng nhà sản xuất
2 nvidia <- data %>% filter(Manufacturer == "Nvidia")
3 amd <- data %>% filter(Manufacturer == "AMD")
4 intel <- data %>% filter(Manufacturer == "Intel")
5 ati <- data %>% filter(Manufacturer == "ATI")
```

4.2.3.a Phương pháp Đồ thị Q-Q

Để kiểm tra phân phối chuẩn, chúng ta sẽ vẽ đồ thị Q-Q cho các nhóm **Nvidia**, **AMD**, **Intel**, và **ATI**. Đồ thị Q-Q giúp so sánh phân phối của dữ liệu với phân phối chuẩn kỳ vọng. Nếu các điểm trên đồ thị Q-Q rơi gần đường chéo, dữ liệu có thể được coi là tuân theo phân phối chuẩn.

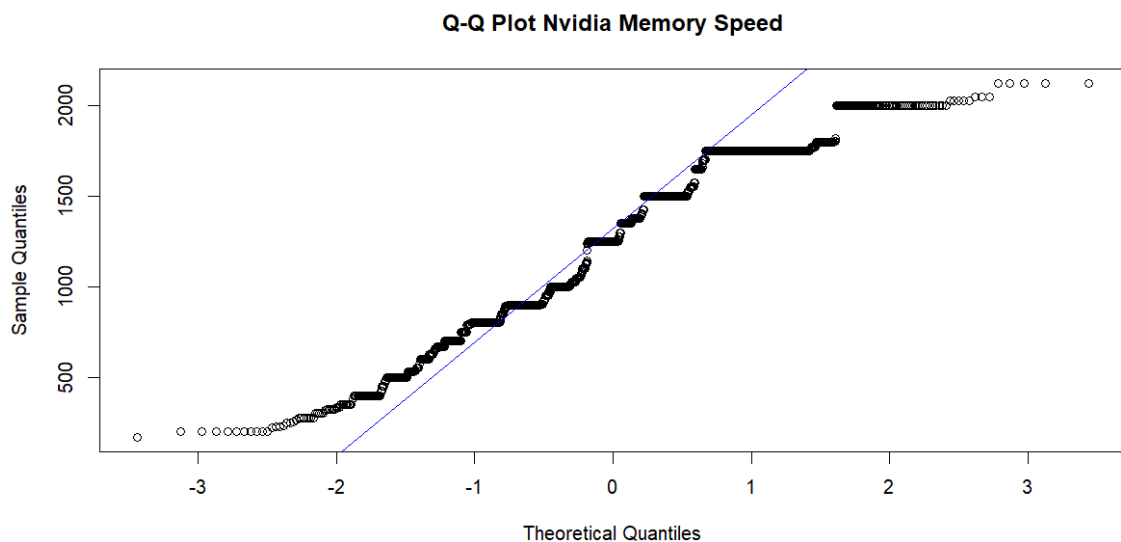
```
1 # Vẽ đồ thị Q-Q cho nhóm Nvidia
2 qqnorm(nvidia$Memory_Speed, main = "Q-Q Plot Nvidia Memory Speed")
3 qqline(nvidia$Memory_Speed, col = "blue")
4
5 # Vẽ đồ thị Q-Q cho nhóm AMD
6 qqnorm(amd$Memory_Speed, main = "Q-Q Plot AMD Memory Speed")
```



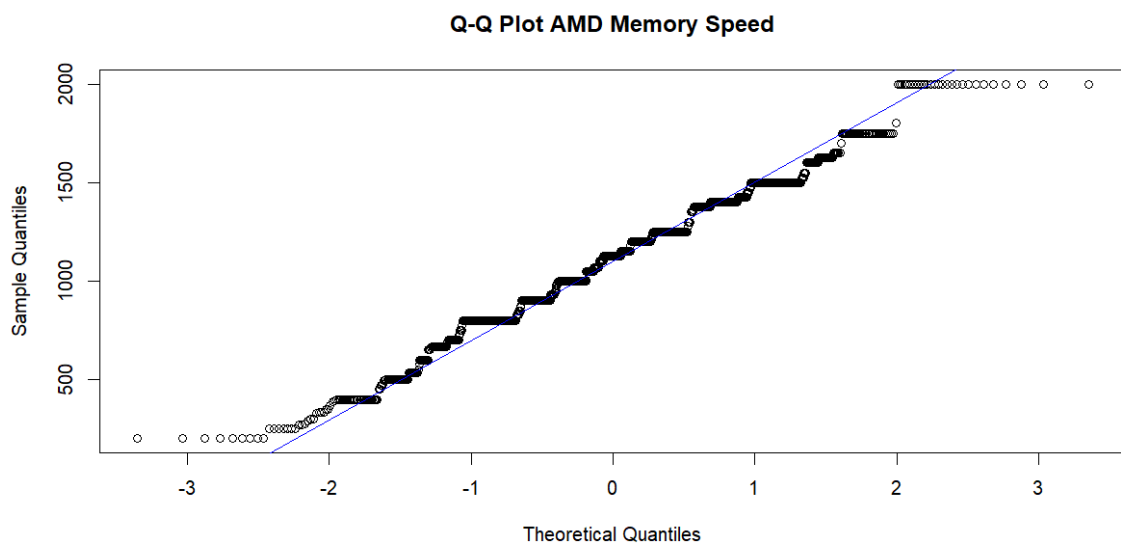
```

7 qqline(amd$Memory_Speed, col = "blue")
8
9 # Vẽ đồ thị Q-Q cho nhóm Intel
10 qqnorm(intel$Memory_Speed, main = "Q-Q Plot Intel Memory Speed")
11 qqline(intel$Memory_Speed, col = "blue")
12
13 # Vẽ đồ thị Q-Q cho nhóm ATI
14 qqnorm(ati$Memory_Speed, main = "Q-Q Plot ATI Memory Speed")
15 qqline(ati$Memory_Speed, col = "blue")

```

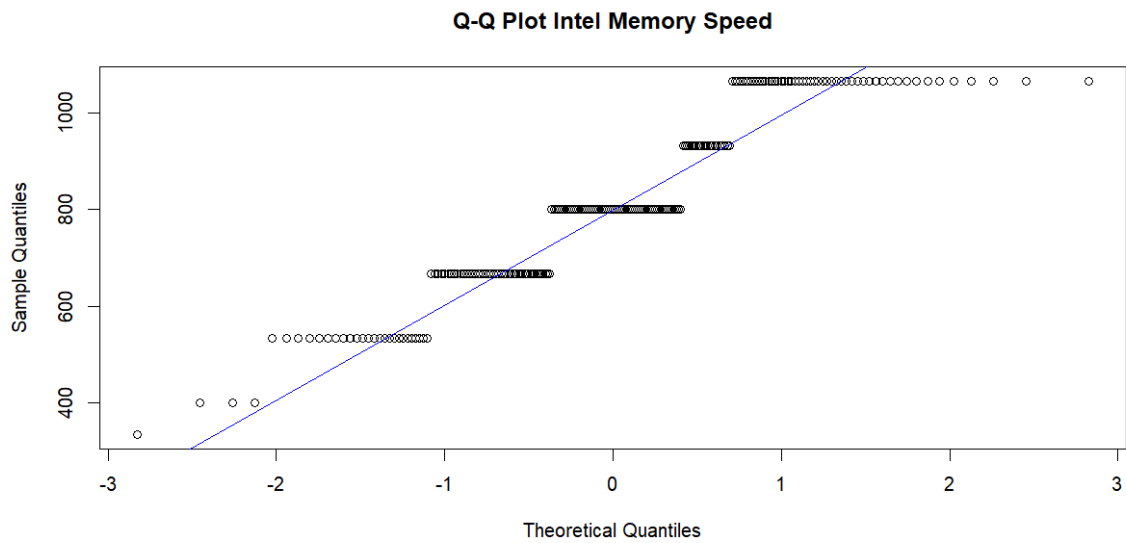


Hình 1: Q-Q Plot cho Nvidia Memory Speed

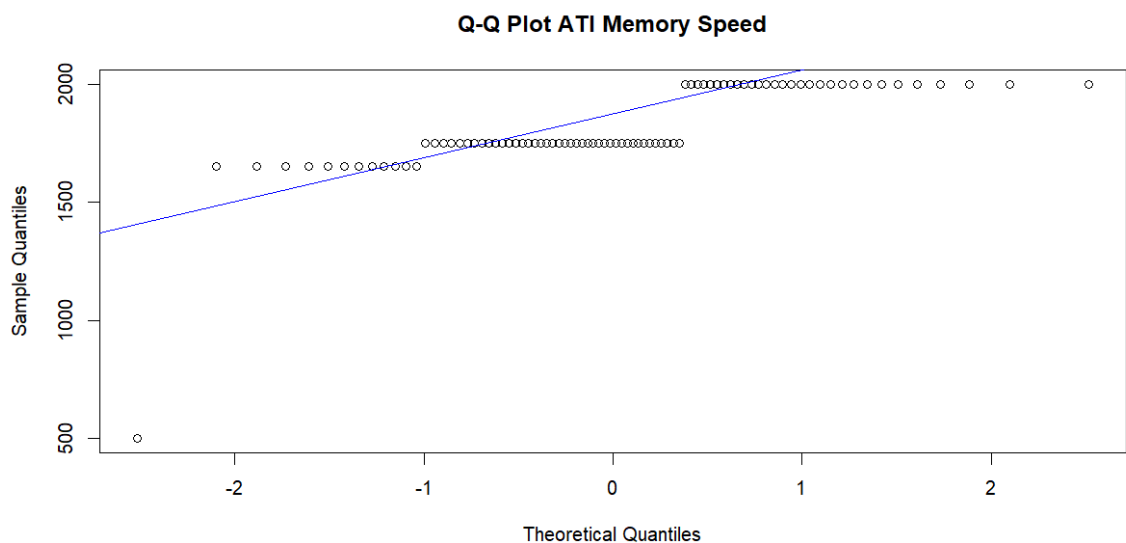


Hình 2: Q-Q Plot cho AMD Memory Speed





Hình 3: Q-Q Plot cho Intel Memory Speed



Hình 4: Q-Q Plot cho ATI Memory Speed

Nhận xét về Kết quả Đồ thị Q-Q: Từ các đồ thị Q-Q, ta có thể nhận thấy các điểm phân phối **Nvidia**, **AMD**, **Intel**, và **ATI** đều không hoàn toàn rơi trên đường thẳng kỳ vọng. Điều này cho thấy rằng dữ liệu của các nhóm **Memory Speed** không hoàn toàn tuân theo phân phối chuẩn, vì các điểm trên đồ thị Q-Q có sự lệch so với đường chéo chuẩn.

4.2.3.b Kiểm định Phương sai Đồng nhất (Levene's Test)

Để kiểm tra sự đồng nhất phương sai giữa các nhóm, ta sử dụng **kiểm định Levene**. Kiểm định này giúp xác định xem các nhóm có phương sai đồng nhất hay không.

Code R để thực hiện kiểm định Levene:



```
1 # Kiểm định phương sai đồng nhất (Levene's Test)
2 library(car)
3 leveneTest(Memory_Speed ~ Manufacturer, data = data)
```

Kết quả Levene's Test:

```
1 Levene's Test for Homogeneity of Variance (center = median)
2      Df F value    Pr(>F)
3 group   4  126.43 < 2.2e-16 ***
4      3200
5 ---
6 Signif. codes:
7 0 '***' 0.001 '**' 0.01 '*' 0.05 '.' 0.1 ' ' 1
```

Nhận xét về Kết quả Levene's Test:

- **Df (Bậc tự do):** Df = 4 cho thấy có bốn nhóm (Nvidia, AMD, Intel, ATI). Đây là số bậc tự do dùng trong kiểm định Levene để xác định sự đồng nhất phương sai giữa các nhóm.
- **F value** = 126.43 là giá trị thống kê F. F-value cao cho thấy sự khác biệt rõ rệt về phương sai giữa các nhóm.
- **p-value** < 2.2e-16 cho thấy p-value rất nhỏ, bác bỏ giả thuyết không H_0 rằng phương sai của các nhóm là đồng nhất. Kết quả này cho thấy phương sai không đồng nhất giữa các nhóm.
- **Signif. codes:** Chỉ số '***' chỉ ra rằng p-value nhỏ hơn 0.001, chứng tỏ sự khác biệt về phương sai giữa các nhóm có ý nghĩa thống kê rất cao.

4.2.4 Giải theo lý thuyết cho ANOVA 1 nhân tố

Trước khi thực hiện phân tích ANOVA bằng hàm `aov()` trong R, chúng ta có thể giải quyết thủ công các bước tính toán bằng các công thức cơ bản sau đây:

- 1. Tính trung bình tổng thể:

$$\text{overall_mean} = \frac{1}{n} \sum_{i=1}^n X_i \quad (1)$$

```
1 overall_mean <- mean(data$Memory_Speed)
```

- 2. Tính trung bình của mỗi nhóm:

$$\text{means_by_group} = \frac{1}{n_k} \sum_{i=1}^{n_k} X_{ik} \quad (2)$$



```
1 means_by_group <- tapply(data$Memory_Speed, data$Manufacturer,
  ↪ mean)
```

• 3. Tính số quan sát trong mỗi nhóm:

$$n_k = \text{Số lượng quan sát trong nhóm } k \quad (3)$$

```
1 n_i <- table(data$Manufacturer)
```

• 4. Tính tổng bình phương sai số giữa các nhóm (SS_between):

$$SS_{\text{between}} = \sum_{k=1}^K n_k (\bar{X}_k - \bar{X})^2 \quad (4)$$

```
1 SS_between <- sum(n_i * (means_by_group - overall_mean)^2)
```

• 5. Tính tổng bình phương sai số trong nhóm (SS_within):

$$SS_{\text{within}} = \sum_{k=1}^K \sum_{i=1}^{n_k} (X_{ik} - \bar{X}_k)^2 \quad (5)$$

```
1 SS_within <- sum(tapply(data$Memory_Speed, data$Manufacturer,
  ↪ function(x) sum((x - mean(x))^2)))
```

• 6. Tính tổng bình phương sai số tổng thể (SS_total):

$$SS_{\text{total}} = \sum_{i=1}^n (X_i - \bar{X})^2 \quad (6)$$

```
1 {SS_total <- sum((data$Memory_Speed - overall_mean)^2)}
```

• 7. Tính các bậc tự do:

$$df_{\text{between}} = K - 1 \quad (7)$$

$$df_{\text{within}} = n - K \quad (8)$$

$$df_{\text{total}} = n - 1 \quad (9)$$



```
1 df_between <- length(means_by_group) - 1
2 df_within <- sum(n_i) - length(means_by_group)
3 df_total <- df_between + df_within
```

• 8. Tính bình phương trung bình:

$$MS_{\text{between}} = \frac{SS_{\text{between}}}{df_{\text{between}}} \quad (10)$$

$$MS_{\text{within}} = \frac{SS_{\text{within}}}{df_{\text{within}}} \quad (11)$$

```
1 MS_between <- SS_between / df_between
2 MS_within <- SS_within / df_within
```

• 9. Tính giá trị F:

$$F = \frac{MS_{\text{between}}}{MS_{\text{within}}} \quad (12)$$

```
1 F_value <- MS_between / MS_within
```

• 10. Tính p-value sử dụng phân phối F:

$$p\text{-value} = 1 - \text{pf}(F, df_{\text{between}}, df_{\text{within}}) \quad (13)$$

```
1 p_value <- 1 - pf(F_value, df_between, df_within)
```

Kết quả tính toán thủ công:

```
1 SS_between: 76877315
2 SS_within: 527945330
3 MS_between: 25625772
4 MS_within: 164982.9
5 F-value: 155.3238
6 p-value: 0e+00
```

4.2.5 Giải bằng hàm ANOVA có sẵn của R

Dưới đây là code sử dụng hàm `aov()` để thực hiện phân tích ANOVA 1 nhân tố.



```
1 # Thực hiện phân tích ANOVA 1 nhân tố
2 anova_result <- aov(Memory_Speed ~ Manufacturer, data = data)
3
4 # Xem kết quả của ANOVA
5 summary(anova_result)
```

Kết quả anova():

```
1      Df      Sum Sq   Mean Sq F value Pr(>F)
2 Manufacturer    4  76877315 25625772   155.3 <2e-16 ***
3 Residuals  3200 527945330   164983
4 ---
5 Signif. codes:
6 0 '***' 0.001 '**' 0.01 '*' 0.05 '.' 0.1 ' ' 1
```

Nhận xét:

- Các giá trị của **SS_between**, **SS_within**, và **MS_between**, **MS_within** trong kết quả thủ công hoàn toàn khớp với kết quả từ hàm `aov()`.
- Giá trị **F-value** cũng trùng khớp, với $F = 155.3238$ tính thủ công và $F = 155.3$ từ hàm `aov()`, khác biệt nhỏ do sự làm tròn trong quá trình tính toán.
- Về p-value, trong trường hợp thủ công, p-value tính toán được là $0e+00$, điều này tương đương với p-value rất nhỏ ($< 2e-16$) trong kết quả từ hàm `aov()`. Điều này cho thấy rằng sự khác biệt giữa các nhóm là rất rõ rệt và có ý nghĩa thống kê cao.
- Tất cả các kết quả từ phương pháp thủ công và phương pháp sử dụng hàm `aov()` đều nhất quán và khớp với nhau. Điều này cho thấy rằng hai phương pháp đều cho kết quả chính xác và đáng tin cậy.

4.2.6 Kết luận

Phân tích ANOVA 1 nhân tố cho phép chúng ta kiểm tra sự khác biệt giữa các nhóm về một biến liên tục như **Memory Speed** trong trường hợp của chúng ta. Qua các bước tính toán và kiểm định, ta có thể kết luận liệu có sự khác biệt có ý nghĩa thống kê giữa các nhà sản xuất GPU hay không.

